NATIONAL RESEARCH UNIVERSITY
HIGHER SCHOOL OF ECONOMICS

Faculty of Computer Science
Bachelor's Programme "HSE and University of London Double Degree Programme in Data Science and Business Analytics"

Programming project report

on the topic Numerical Algorithms of Linear Algebra

Student:
group БПАД232 _____ З. Ю. Зинкин
Sign Name, Patronymic, Surname
4.02.2025
Date

Supervisor: Дмитрий Витальевич Трушин
Name, Patronymic, Surname
доцент / ФКН НИУ ВШЭ
Position / Company
4.02.2025 _____ _____
Date Grade Sign

Moscow 2025

Contents

Abstract

The main focus of the programming project is to study matrix decomposition algorithms from a theoretical point of view with its subsequent implementation on the C++ language.

1 Introduction

Decomposition algorithms are extensively used in Numerical Linear Algebra and Computer Science. They can help store large matrices, find eigenvalues of matrices, solve systems of linear equations and linear least squares problems and much more. This is why it is vital not only to recognize the potential use cases of decomposition algorithms but also understand the underlying theory and implementation of such algorithms.

The goal of my programming project is to learn modern algorithms of Numerical Linear Algebra with a focus on matrix decomposition algorithms, which represent a matrix in a certain way, as a product of other matrices, for further computation. In order to have a profound understanding of each of the algorithms, it is necessary to implement a C++ library from scratch.

The library itself contains a class for matrix representation with implementation of matrix arithmetic and other necessary methods. Decomposition algorithms are implemented based on the aforementioned matrix class. The whole library will is tested using Googletest[2].

The information for the theoretical part of the project is taken from the books, which are cited at the end of the paper. Citation of exact chapters is used throughout the theoretical part when needed.

The result of the project is a C++ library LinAlgTools[3] with the following functionality:

- Matrix arithmetic
- QR Decomposition
- SVD Decomposition

Theoretical part can help you understand the underlying theory of each of the algorithms. The technical part of the project is described in detail in the Library LinAlgTools section of the paper. The results of the testing using Googletest[2] are documented in the Testing section.

2 Description of functional and non-functional requirements for the program project

2.1 Functional requirements

- A *Matrix* class, which allows users to perform matrix arithmetic. The class must be able to support both real and complex elements. Moreover, the class must have a clear and intuitive interface.
- A *SubMatrix and ConstSubMatrix* classes for handling a particular part of an existing matrix without copying data. These classes must have the same methods as the *Matrix* class in order for users to be able to work with them in the same manner, as with regular matrices. Furthermore, cross-type arithmetic must be implemented to perform arithmetic between objects of *Matrix* and *SubMatrix* classes of appropriate sizes.
- Implementation of numerically stable algorithms:
  - QR Decomposition using Householder rotations - representation of a matrix as a product of Q and R matrices, which are computed using Householder rotations.
  - QR Decomposition using Given's rotations - representation of a matrix using specific Q and R matrices, which are computed using Givens rotations.
  - SVD Decomposition - representation of a matrix as a product of U, Σ and V^* matrices, where U, V are unitary and Σ is a diagonal matrix.
- Extensive testing must be done to guarantee proper functioning of the library. Randomized performance tests are to be analyzed and compared with other existing libraries.

2.2 Non-functional requirements

- Implementation of the library on the C++ language without third-party libraries, except for Googletest.
- *Git*, a version control system, and a remote repository on *GitHub* [3].
- *clang* compiler with C++23 standart support.
- Code formatter *clang-format* to automatically format code based on the Microsoft style with additional changes.
- Static code analyzer *clang-tidy* to identify potential errors and to stick to uniform naming of variables, functions, etc.
- A third-party library *Googletest* [2] for writing tests.
- A text editor with IDE capabilities *Neovim*
- Minimum RAM requirement: 4GB.

3 Theorerical part

3.1 Main definitions

Definition 1. An $m \times n$ *matrix* over a field $\mathbb{F}$ (either $\mathbb{C}$ or $\mathbb{R}$) is a rectangular array of elements of $\mathbb{F}$, arranged in m rows and n columns:

$$\begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix}$$

A set of all $m \times n$ matrices over a field $\mathbb{F}$ we denote $\mathbb{F}^{m \times n}$.

Definition 2. A matrix $A \in \mathbb{F}^{m \times n}$ is called *diagonal* if all its elements not on the diagonal are 0:

$$\begin{bmatrix} a_{11} & 0 & 0 & \dots & 0 & \dots & 0 \\ 0 & a_{22} & 0 & \dots & 0 & \dots & 0 \\ 0 & 0 & a_{33} & \dots & 0 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & a_{mk} & \dots & 0 \end{bmatrix}$$

A special case of a diagonal matrix is an $n \times n$ *Identity* matrix:

$$I_n = \begin{bmatrix} 1 & 0 & 0 & \dots & 0 \\ 0 & 1 & 0 & \dots & 0 \\ 0 & 0 & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & 1 \end{bmatrix}$$

Definition 3. A matrix $A \in \mathbb{F}^{m \times n}$ is called *upper triangular* if all its elements below the diagonal are 0:

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & \dots & a_{1k} & \dots & a_{1n} \\ 0 & a_{22} & a_{23} & \dots & a_{2k} & \dots & a_{2n} \\ 0 & 0 & a_{33} & \dots & a_{3k} & \dots & a_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & a_{mk} & \dots & a_{mn} \end{bmatrix}$$

Definition 4. A vector is an $1 \times n$ or $n \times 1$ matrix:

$$\begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix} \text{ or } \begin{bmatrix} a_1 & a_2 & \dots & a_n \end{bmatrix}$$

A set of all $m \times 1$ or $1 \times m$ vectors over $\mathbb{F}$ we denote $\mathbb{F}^m$.

Definition 5. A *Hermitian conjugate* of a matrix $A \in \mathbb{C}^{m \times n}$ is an $n \times m$ matrix, obtained by transposing A and applying complex conjugate to each of entry: $(A^*)_{ij} = \overline{A}_{ji}$.

Remark. For real matrices *Hermitian conjugate* coincides with transposition of the matrix: $A^* = A^T$.

Definition 6. A matrix $A \in \mathbb{C}^{m \times n}$ is called *Hermitian* if $A^* = A$.

Definition 7. A matrix $A \in \mathbb{C}^{m \times n}$ is called *Unitary* if $A^* = A^{-1}$. As a consequence, $A^*A = AA^* = I$.

Definition 8. A matrix $A \in \mathbb{R}^{m \times n}$ is called *Orthogonal* if $A^T = A^{-1}$. As a consequence, $A^TA = AA^T = I$.

Statement 1. *A product of unitary (orthogonal) matrices is also unitary (orthogonal).*

Proof. Let U and V be unitary:

$$(UV)^*UV = V^*U^*UV = V^*V = I$$

□

Definition 9. A *scalar product* is a bilinear form $\langle \cdot, \cdot \rangle : \mathbb{F}^n \times \mathbb{F}^m \to \mathbb{F}$. In a Euclidean space, such as $\mathbb{R}^m$ or $\mathbb{C}^m$ the scalar product of two vector $v, w \in \mathbb{C}^m$ is defined in the following way:

$$\langle v, w \rangle = w^* v = \sum_{i=1}^{m} v_i \overline{w_i}$$

Definition 10. A 2-norm, or Euclidean norm, on an Euclidean space is a function: $\|\cdot\|_2 : \mathbb{F}^n \to \mathbb{R}_+$.

Let $v \in \mathbb{F}^n$:

$$\|v\|_2 = \sqrt{\langle v, v \rangle} = \sqrt{\sum_{i=1}^{m} |v_i|^2}$$

Geometrically, a 2-norm returns the distance from the origin to the point in the space.

3.2 Algorithms

3.2.1 Householder reflection

A *Householder reflection* is a linear transformation which reflects a vector about a hyperplane.[1]

Given a vector $v \in \mathbb{F}^m : \|v\| = 1$, an $m \times m$ matrix P (or more commonly $H(v)$) of the form

$$P = H(v) = I - 2vv^*$$

is called a *Householder reflector* or *Householder matrix*. The matrix allows us to reflect vector along the hyperplane, spanned by v. Moreover, a Householder matrix is unitary and hermitian ($H^{-1} = H^* = H$), which turns out to be key in the QR decomposition algorithm.

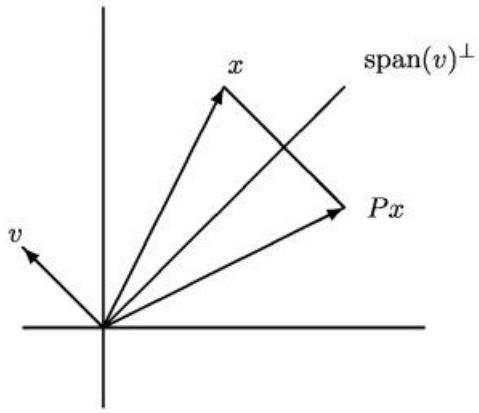

Figure 1: Householder reflection applied to a vector x

Statement 2. *$\forall a, b \in \mathbb{C}^n : \|a\|_2 = \|b\|_2$, $\exists \gamma \in \mathbb{C} : |\gamma| = 1$ and $v \in \mathbb{C}^n : \|v\|_2 = 1$, such that $H(v)a = \gamma b$.*

Proof. Using direct computations we can find the required γ and v:

$$H(v) = I - 2vv^*$$

$$H(v)a = a - 2vv^*a = \gamma b$$

$$2(v^*a)v = a - \gamma b \tag{1}$$

From here it follows that v must be of the form:

$$v = \mu \frac{a - \gamma b}{\|a - \gamma b\|_2}$$

But since we are proving the existence, we can set μ to 1, hence:

$$v = \frac{a - \gamma b}{\|a - \gamma b\|_2}$$

Plugging it into (1):

$$2\frac{(a-\gamma b)^*}{\|a-\gamma b\|_2} a \frac{a-\gamma b}{\|a-\gamma b\|_2} = a - \gamma b$$

$$2\frac{(a-\gamma b)^*}{\|a-\gamma b\|_2} a \frac{1}{\|a-\gamma b\|_2} = 1$$

$$2(a-\gamma b)^* a = \|a-\gamma b\|_2^2$$

$$2(a-\gamma b)^* a = (a-\gamma b)^*(a-\gamma b)$$

$$2a^*a - 2\overline{\gamma} b^* a = a^*a + b^*b - \gamma a^* b - \overline{\gamma} b^* a$$

$$\overline{\gamma} b^* a = \mathrm{Re}(\overline{\gamma} b^* a)$$

$$\gamma = \pm \frac{b^* a}{|b^* a|}$$

If $a \perp b$, γ can be any number, such that $|\gamma| = 1$. If $a \parallel b$, then we should carefully choose the sign to guarantee numeric stability of the transformation. □

Corollary 1. $\forall x \in \mathbb{C}^n \; \exists v \in \mathbb{C}^n : \|v\|_2 = 1$ *and*

$$H(v)x = \gamma \begin{bmatrix} \|x\|_2 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

$$v = \frac{x - \gamma \|x_2\|_2 e_1}{\|x - \gamma \|x\|_2 e_1\|_2}, \; \gamma = -\frac{x_1}{|x_1|}$$

The corollary yields an algorithm for transforming a vector, and if we want to apply the transformation to a matrix from the left (right), we can use:

Algorithm 1 Householder Left Reflection

```
function HouseholderLeftReflection(A ∈ ℂ^{m×n}, x ∈ ℂ^m)
    v = x
    v_1 = x_1 − sign(x_1) · ‖x‖_2
    v = v · 1/‖v‖
    M = A − (2v) · (v* A)
    return M
end function
```

Algorithm 2 Householder Right Reflection

```
function HouseholderRightReflection(A ∈ ℂ^{m×n}, x* ∈ ℂ^n)
    v = x
    v_1 = x_1 − sign(x_1) · ‖x‖_2
    v = v · 1/‖v‖
    M = A − (Av*) · (2v)
    return M
end function
```

3.2.2 Givens rotations

In cases when we need to zero specific elements of the matrix, *Givens rotations* are our tool of choice. A *Givens rotation* is a linear transformation which amounts to a counterclockwise rotation by an angle of θ. The matrix of a *Givens rotation* is of the form:

$$G(i,j,\theta) = \begin{bmatrix} 1 & \cdots & 0 & \cdots & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & & \vdots & & \vdots \\ 0 & \cdots & \cos\theta & \cdots & -\sin\theta & \cdots & 0 \\ \vdots & & \vdots & \ddots & \vdots & & \vdots \\ 0 & \cdots & \sin\theta & \cdots & \cos\theta & \cdots & 0 \\ \vdots & & \vdots & & \vdots & \ddots & \vdots \\ 0 & \cdots & 0 & \cdots & 0 & \cdots & 1 \end{bmatrix} \begin{matrix} \\ \\ i \\ \\ j \\ \\ \\ \end{matrix}$$
$$\qquad\qquad\qquad\qquad\quad i \qquad\quad j$$

Notice that G is almost the identity matrix. We can exploit this structure to achieve faster computation by apply the transformation directly to the rows which are affected by G.

Algorithm 3 Givens Left Rotation

function GivensLeftRotation($A \in \mathbb{F}^{m\times n}$, $i \in \mathbb{N}_+$, $j \in \mathbb{N}_+$, $a \in \mathbb{F}$, $b \in \mathbb{F}$)
  $r = \sqrt{a^2 + b^2}$
  $c = a/r$
  $s = -b/r$
  $M = A$
  for k = 1 to n **do**
    $M_{ik} = \overline{c} \cdot A_{ik} - \overline{s} \cdot A_{jk}$
    $M_{jk} = s \cdot A_{ik} + c \cdot A_{jk}$
  end for
  return M
end function

Algorithm 4 Givens Right Rotation

function GivensRightRotation($A \in \mathbb{F}^{m\times n}$, $i \in \mathbb{N}_+$, $j \in \mathbb{N}_+$, $a \in \mathbb{F}$, $b \in \mathbb{F}$)
  $r = \sqrt{a^2 + b^2}$
  $c = a/r$
  $s = -b/r$
  $M = A$
  for k = 1 to m **do**
    $M_{ki} = c \cdot A_{ki} - s \cdot A_{kj}$
    $M_{kj} = \overline{s} \cdot A_{ki} + \overline{c} \cdot A_{kj}$
  end for
  return M
end function

3.2.3 QR decomposition

Statement. For any matrix $A \in \mathbb{F}^{m\times n}$ exist a unitary matrix $Q \in \mathbb{F}^{m\times m}$ and an upper-triangular matrix $R \in \mathbb{F}^{m\times n}$, such that $A = QR$.

Proof. Using Householder rotations (for convenience $A \in \mathbb{F}^{5\times 3}$):

$$A = \begin{bmatrix} \star & \star & \star \\ \star & \star & \star \\ \star & \star & \star \\ \star & \star & \star \\ \star & \star & \star \end{bmatrix} \overset{H(v_1)}{\to} \begin{bmatrix} \star & \star & \star \\ 0 & \star & \star \\ 0 & \star & \star \\ 0 & \star & \star \\ 0 & \star & \star \end{bmatrix} \overset{H(v_2)}{\to} \begin{bmatrix} \star & \star & \star \\ 0 & \star & \star \\ 0 & 0 & \star \\ 0 & 0 & \star \\ 0 & 0 & \star \end{bmatrix} \overset{H(v_3)}{\to} \begin{bmatrix} \star & \star & \star \\ 0 & \star & \star \\ 0 & 0 & \star \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} = R$$

$$H_3H_2H_1A = R$$
$$A = H_1^{-1}H_2^{-1}H_3^{-1}R = H_1H_2H_3R = QR$$

□

The proof yields an algorithm:

Algorithm 5 Householder QR Decomposition

```
function HouseholderQR(A ∈ F^{m×n})
    Q = I_m
    R = A
    for k = 1 to n do
        x = R[k : m, k]
        v = HouseholderReduction(x)
        R[k : m, k : n] = HouseholderLeftReflection(R[k : m, k : n], v)
        Q[k : m, 1 : m] = HouseholderLeftReflection(Q[k : m, 1 : m], v)
    end for
    Q = Q^T
    return Q, R
end function
```

Proof. Using Givens rotations (for convenience $A \in \mathbb{F}^{3\times 2}$):

$$A = \begin{bmatrix} \star & \star \\ \star & \star \\ \star & \star \end{bmatrix} \overset{G_{13}}{\to} \begin{bmatrix} \star & \star \\ \star & \star \\ 0 & \star \end{bmatrix} \overset{G_{12}}{\to} \begin{bmatrix} \star & \star \\ 0 & \star \\ 0 & \star \end{bmatrix} \overset{G_{23}}{\to} \begin{bmatrix} \star & \star \\ 0 & \star \\ 0 & 0 \end{bmatrix} = R$$

$$G_{23}G_{12}G_{13}A = R$$
$$A = G_{13}^*G_{12}^*G_{23}^*R = QR$$

□

The proof once again yields an algorithm:

Algorithm 6 Givens QR Decomposition

```
function GivensQR(A ∈ F^{m×n})
    Q = I_m
    R = A
    for k = 1 to n do
        for i = m downto k + 1 do
            a = R[i − 1, j]
            b = R[i, j]
            if b ≠ 0 then
                R[1 : m, k : n] = GivensLeftRotation(R[1 : m, k : n], i − 1, i, a, b)
                Q[1 : m, 1 : m] = GivensRightRotation(Q[1 : m, 1 : m], i − 1, i, a, b)
            end if
        end for
    end for
    return Q, R
end function
```

4 Library LinAlgTools

This section of the paper will contain description of the library's architecture, code references and examples of library's usage.

5 Testing

In order to ensure the proper functioning of the library, Googletest [2] is used. This section of the paper will contain results of the tests with additional performance graphs.

6 Conclusion

This section of the paper will contain the results of the development of the programming project with a brief overview of the accomplished work.

References

[1] Charles F. Van Loan Gene H. Golub. Matrix computations, 4th edition. URL: https://disk.yandex.ru/i/11ZWE2ud-IZtMA, 2013.

[2] Google. Googletest. URL: https://github.com/google/googletest, 1.15.2.

[3] Zinkin Zakhar. LinAlgTools. URL: https://github.com/Avgustineiw/LinAlgTools.